Lahore University of Management Sciences
School of Science and Engineering
Department of Artificial Intelligence

Programming Assignment 2
Questions 1 and 2

River Forecasting and Moving MNIST

Course	AI651 – Deep Learning for Space, Time and Graphs
Student	Muhammad Bin Tariq
Roll number	27100017
GitHub repository	https://github.com/MuhammadBinTariq/DL4STG-PA2
Semester	Fall 2026

Question 1 — River Forecasting Proposal

1. Architecture diagram

Historical inputs available at issuance t_0
Water levels: S1–S4 and T1 $(B, 5, 18, 3)$
Rainfall: R1 and R2 $(B, 2, 18, 3)$
Reservoir: actual and historical scheduled releases $(B, 1, 18, 6)$
Align to ten-minute intervals; normalize readings; retain masks and ages.

↓

Project and combine the sources
Separate linear projections to width 32; stack eight locations.
Add location/type features $(8, 6)$ and time-position embeddings.
Hidden tensor: $(B, 8, 18, 32)$.

↓

Five spatial–temporal blocks
Spatial attention: $(B \cdot 18, 8, 32)$; mix locations at each time.
Restore axes, then temporal convolution: $(B \cdot 8, 32, 18)$.
Kernel 5, left padding 4; mix past times at each location.
ReLU and residual connections; restore $(B, 8, 18, 32)$.

↓

Forecast head
Select latest S1–S4 features: $(B, 4, 32)$.
Known future release values and masks: $(B, 6, 2)$
$\rightarrow$ flatten to $(B, 12)$ and repeat across four stations.
Concatenate to $(B, 4, 44)$; shared MLP $44 \rightarrow 32 \rightarrow 1$.
Water levels at $t_0 + 60$ minutes: $(B, 4)$.

We use spatial attention and temporal convolution. The locations are irregularly spaced, so we do not treat them as a regular spatial grid. Attention combines their information while preserving a separate representation for each location. Temporal convolution then combines recent observations.

2. Assumptions

The supplied facts are the measured locations, river connections and distances, ten-minute water-level records, timestamped rainfall, and reservoir release records and announcements. T1 lies on a tributary joining between S1 and S2; the unmonitored tributary joins between S2 and S3. Precise travel times and future rainfall are unavailable.

A1: History length. We initially use 18 observations ending at issuance, from $t_0 - 170$ minutes to t_0. This gives a short window of recent conditions without a large model. We would compare longer windows on validation data; we do not assume all relevant travel times fit within this window.

A2: Historical coverage. We assume one to two years of records, including wet and dry periods, can be provided. Repeated seasonal conditions would support training, but the number of independent flood events matters more than the number of overlapping windows. Limited flood coverage would reduce confidence in extreme-event forecasts.

A3: Timing and quality. We assume timestamps, units and station identifiers can be checked, and arrival times or reporting-delay logs can be obtained. We also assume rainfall units permit conversion to ten-minute totals. Invalid readings are marked missing. If historical arrival logs are unavailable, validation would simulate plausible delays and could not fully reproduce past operational conditions.

A4: Physical influence. We expect reservoir releases and upstream conditions to influence downstream levels, with delays that vary by location and river conditions. We do not prescribe exact delays or assume the same delay per station. Locations and river distances identify these differences; the model learns their predictive relationships.

A5: Rainfall coverage. We assume R1 and R2 contain some useful information about recent catchment rainfall. Their coverage of the unmonitored tributary is uncertain. Thus, rainfall is an indirect predictor of its effect, not a measurement of its inflow.

A6: Stability between updates. We assume historical relationships remain useful over the next hour under ordinary conditions. Major channel changes, unusual floods or reservoir operating changes may violate this assumption. These require monitoring and possible retraining.

3. Forecasting task and data specification

Every ten minutes, at issuance time t_0, we predict water levels at S1–S4 at $t_0 + 60$ minutes. The horizon is six sampling intervals, but the output contains only the four levels at that final time.

Historical inputs must have both measurement and arrival times no later than t_0. Future release inputs use only schedules announced by t_0. Historical schedule values retain the versions announced before their effective intervals, so differences from actual releases reflect genuine operational discrepancies. Future rainfall and future measured levels are excluded.

Water levels retain their ten-minute timestamps. Rainfall is accumulated over each preceding ten-minute interval, and releases are averaged over that interval. Different measurement types are normalized using training-set statistics. The location order is $[U, S1, T1, S2, R1, R2, S3, S4]$; this is an indexing convention, not a claim that all locations lie along one river line.

Input/output	Dimensions	Axes, channels and time coverage
Water-level history	$(B, 5, 18, 3)$	Batch, locations S1–S4/T1, time, channels. Channels: value, availability mask, age since last available valid reading. Times $t_0 - 170, \ldots, t_0$ minutes.
Rainfall history	$(B, 2, 18, 3)$	Batch, gauges R1/R2, time, channels. Same channel meanings; values are preceding ten-minute totals ending at each historical timestamp.
Reservoir history	$(B, 1, 18, 6)$	Batch, reservoir, time, channels. Separate value/mask/age triplets for actual and historical scheduled releases over the same intervals.
Future releases	$(B, 6, 2)$	Batch, six future intervals, channels: scheduled rate and availability mask. Intervals $(t_0+10(h-1), t_0+10h]$, $h = 1, \ldots, 6$, in minutes; announcements available by t_0.
Location features	$(8, 6)$	Location, features: two coordinates, one river-distance coordinate and three type indicators (level, rain, release). Shared across batches and times. Rain gauges use the river coordinate of the nearest mapped main-river point as a location descriptor, not a runoff route.
Time positions	$(18, 32)$	Historical position, learned embedding features. Shared across batches and locations; positions refer only to the observed history.
Forecast/target	$(B, 4)$ each	Batch, stations S1–S4. Water levels in metres at $t_0 + 60$ minutes. Targets are used only for training and evaluation.

Availability is one only for a valid value available at issuance. Ages are scaled and capped; a missing initial history uses the cap. Separate projections accommodate the reservoir's six channels and the other sources' three channels before stacking. Coordinates and distances are scaled. The river-distance coordinate is measured along the mapped network from a fixed reference; T1's coordinate includes its tributary segment. It is a position descriptor, not a measured travel time.

4. Model computations and design justification

Let i index locations and j index historical times. Let $x_{i,j}$ contain the source's value, mask and age channels, and let g_i contain its six location features. Using a projection $P_{s(i)}$ for each source type, the initial representation is

$$H^0_{i,j} = P_{s(i)} x_{i,j} + b_{s(i)} + W_g g_i + p_j.$$

Each of five blocks has its own weights. At each fixed time, spatial attention mixes the eight locations:

$$Q_j = H^{\ell-1}_{:,j} W^\ell_Q, \quad K_j = H^{\ell-1}_{:,j} W^\ell_K, \quad V_j = H^{\ell-1}_{:,j} W^\ell_V,$$

$$Z^\ell_{:,j} = H^{\ell-1}_{:,j} + \mathrm{ReLU}\left(\mathrm{softmax}_{\mathrm{row}} \left(\frac{Q_j K_j^\top}{\sqrt{32}} \right) V_j \right).$$

At each fixed location, a temporal convolution uses the current and four preceding positions:

$$H^{\ell}_{i,j} = Z^{\ell}_{i,j} + \mathsf{ReLU}\left(b^{\ell}_{T} + \sum_{r=0}^{4} A^{\ell}_{r} Z^{\ell}_{i,j-r} \right), \qquad \ell = 1, \ldots, 5.$$

Positions before the window are zero-padded. Temporal weights are shared across locations; spatial weights are shared across times. Five kernel-5 convolutions provide a receptive field of $1+5(5-1) = 21$ positions, covering all 18 available observations. This is information access, not a guarantee that the model learns the correct travel delays.

For each target station, concatenate its latest representation with the flattened future schedule and mask, $v \in \mathbb{R}^{12}$:

$$\widehat{y}_i = w_2^{\top}\, \mathsf{ReLU}\left(W_1 [H^{5}_{i,18} \| v] + b_1 \right) + b_2.$$

The head is shared across the four stations. Their distinct representations allow the same announced release to have different predicted effects. Predictions and targets are in metres. Training minimizes squared error over available target readings:

$$\mathcal{L} = \frac{\sum_{b,i} a_{b,i} (\widehat{y}_{b,i} - y_{b,i})^2}{\sum_{b,i} a_{b,i}},$$

where $a_{b,i}$ marks a valid target; batches with no valid targets are skipped.

Why this combination. Spatial attention suits irregular locations and allows their contributions to depend on observed conditions. Temporal convolution combines several past lags. Alternating the operations lets later blocks combine information already drawn from both locations and times, while keeping the location axis until the forecast head. These are the spatial attention and temporal convolution operators, combined through block-level factorization.

How the sources contribute. For S1, reservoir history and announced releases provide evidence of incoming changes. For S2, S1 and T1 describe upstream and monitored tributary conditions. For S3, S2 and rainfall provide evidence of upstream changes and possible unmonitored inflows. S4 can combine these with its own and S3's recent levels. All stations can also use other observed locations when these provide useful context.

River connectivity describes expected physical influences; it is not used as an adjacency mask. Attention weights are learned feature weights, not proportions of water. Observed downstream levels may help predict upstream levels through shared conditions without implying upstream water transport. Variable delays may be represented through several temporal lags and condition-dependent features, but are not explicitly measured or guaranteed by this design.

We would split the record chronologically into training, validation and test periods, keeping complete history/target windows within each period. History length and model size would be selected using validation data. Evaluation would report errors at each station and during high-water events, including comparison with holding the latest observed level constant.

5. Operation and exceptional conditions

(i) Missing observations. For water levels and actual releases, use the last available valid value, with mask zero and increasing age. For missing rainfall, use a zero placeholder with mask zero; this does not mean no rain occurred. If no earlier reading exists, use the training mean as a placeholder. Missing historical schedules are also marked explicitly. Training would include simulated gaps and delays. Prolonged outages trigger a degraded-data warning; the authority would set the acceptable age limits before deployment.

(ii) Delayed observations. At t_0, only arrived readings enter the historical buffer. Late readings are treated as missing for that issuance. Once received, they update the buffer for the next forecast, without changing what was available to earlier forecasts.

(iii) Announced future releases. For each future interval, use the latest schedule version announced by t_0 that covers that interval. Uncovered intervals hold the last available actual release as a fallback, with availability zero. If that is also unavailable, use a training-mean placeholder and flag the missing release information.

(iv) Scheduled versus actual releases. Historical actual and scheduled values remain separate inputs. Their differences may help the model adjust its use of the future schedule. The announced future rate remains a plan, not an observed release; an unexpected override after issuance cannot be known in advance.

Operational feasibility. Each run processes eight locations, 18 historical positions and five small blocks. This is a modest workload for a ten-minute service. End-to-end latency, including data collection, would be checked before deployment; no measured runtime is claimed here.

6. Uncertainty, limitations, and additional information

Unmonitored inflows and future rain. Local storms can raise S3/S4 without being captured by R1/R2 or the upstream gauges. Recent rainfall offers indirect evidence, but future rain remains unknown. Forecasts would be refreshed every ten minutes; this reduces the time before new observations are incorporated, but cannot prevent the initial error.

Changing travel times and rare events. Multiple lags may capture changes seen in training. Longer delays, unusual floods and altered channels can still cause mistimed peaks. We would monitor station-level and high-water errors, test longer histories, and retrain when relationships change.

Uncertainty reporting. We would form station-specific empirical intervals from held-out chronological residuals and check their coverage during ordinary and high-water conditions. Nominal 90% coverage would not be presented as guaranteed during unseen extremes or major outages.

Additional information. A gauge on the unmonitored tributary would provide direct level observations; a rating curve would also be needed to estimate discharge. Catchment boundaries and additional rain gauges would clarify rainfall coverage. Arrival logs and archived release announcements would improve realistic validation. Reservoir levels and gate records would help explain release discrepancies. The system remains usable without these additions, but retains the corresponding inflow, coverage, timing or operational uncertainty. Rainfall nowcasts available at issuance could further help if later supplied; the proposed model does not assume their availability.

1 Setup and Training Motion

Moving MNIST uses a fixed 8,000/1,000/1,000 train/validation/test video split, with twenty 64×64 grayscale frames per video. Five inputs forecast five frames; at $3\times$ and start 1, only two targets exist. All learned models use Adam at 10^{-3}, batch size 32, 30 epochs, final weights and seeds 0/1 on an NVIDIA A100-SXM4-40GB. Loss weights videos equally after averaging their valid targets. Tests use all 1,000 videos at start 1; entries are mean (seed 0, seed 1). $E^{(5)}$ and $E^{(2)}$ average five and two horizons respectively.

$$C_T = \mathrm{MSE}(f(TX), T(f(X))), \qquad E_T = \mathrm{MSE}(f(TX), T(Y)).$$

Table 1. Transformation and window checks. Marker locations are zero-based; window frame numbers are one-based.

Check	Expected	Measured
(a) max $\lvert M(M(x)) - x\rvert$	0	0.0
(b) max $\lvert R^4(x) - x\rvert$	0	0.0
(c) max $\lvert MRM(x) - R^{-1}(x)\rvert$	0	0.0
(d) Bright pixel of $M(p)$	(7, 10, 43)	(7, 10, 43)
(e) Bright pixel of $R(p)$	(7, 43, 10)	(7, 43, 10)
(f) $1\times$ inputs / targets	1, 2, 3, 4, 5 / 6, 7, 8, 9, 10	1, 2, 3, 4, 5 / 6, 7, 8, 9, 10
(g) $2\times$ inputs / targets	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h) $3\times$ inputs / targets	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

All three group-relation checks passed with exact equality. Optical flow uses the specified Farneback settings on the first 50 training videos.

Q2.1: Training motion statistics

Figure 1(a) shows samples in every direction range, but the bars have different heights. Thus, directions are broadly represented without being equally common. Mirroring or rotation can change their relative frequencies. The transformed motion directions would therefore likely be familiar, although this does not guarantee successful forecasting.

From Figure 1(b), a typical speed is 3-4 pixels/frame, with a median of 3.32. Scaling the unrounded median gives nominal references of 6.64 and 9.95 pixels per observed step at $2\times$ and $3\times$. These are scaled references, not separately measured flow medians.

Where digits overlap, one displacement vector may not represent either digit's motion accurately. This could bias both histograms and the median. The size of this bias was not quantified, so the conclusions remain tentative.

Q2.2: Effects of the transformations

Mirroring changes digit appearance and reverses the horizontal motion component; rotation changes appearance and turns each direction by $90°$. Both preserve speed, so the speed histogram should remain largely unchanged while direction frequencies may change.

Speed-up retains existing frames at larger intervals. It preserves digit shape but increases displacement between observations.

Together, these provide useful controlled comparisons: mirroring/rotation test appearance and orientation changes at unchanged speed, while speed-up tests larger movements at unchanged shape. However, mirroring and rotation do not isolate appearance from motion direction.

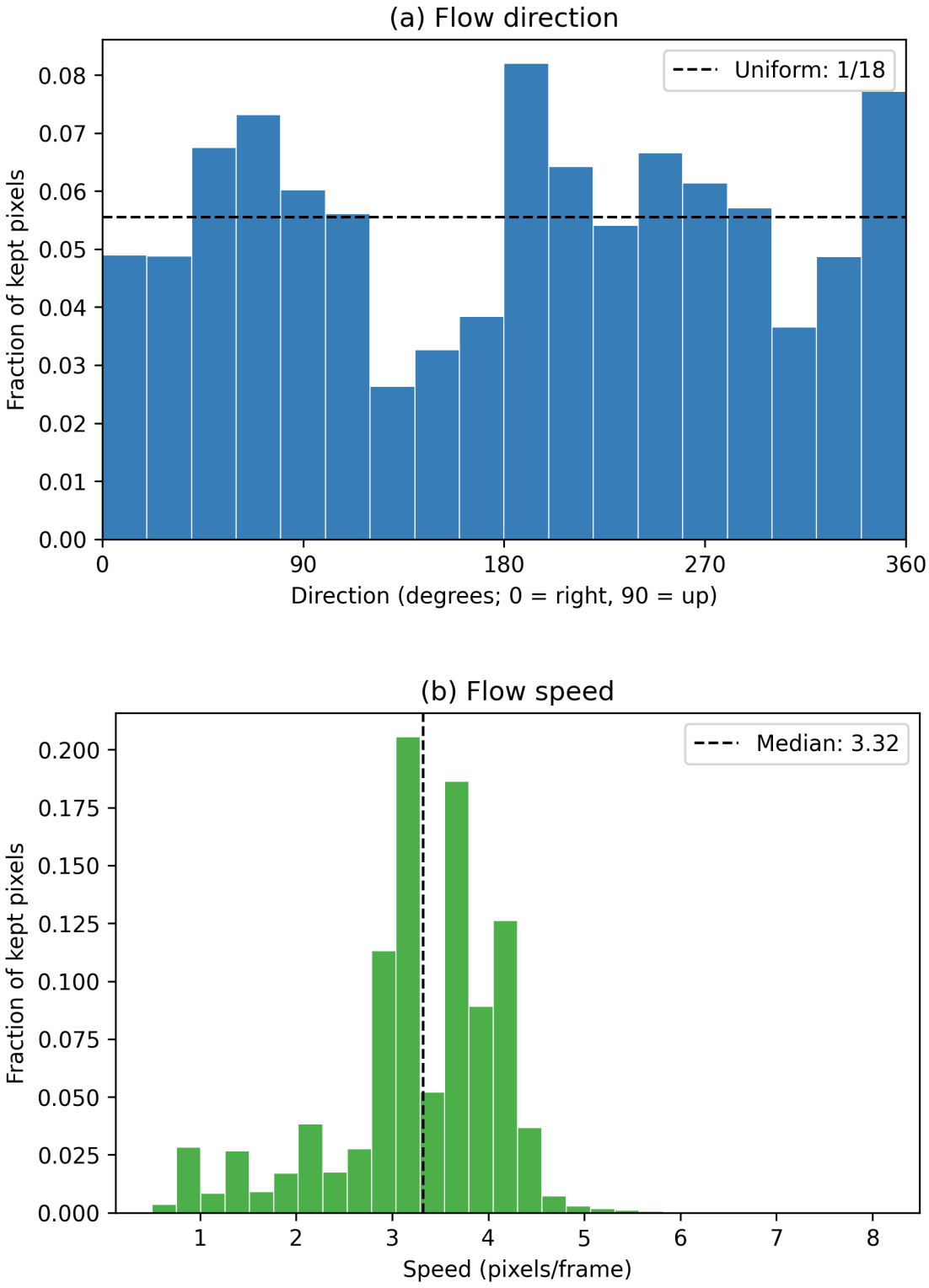


Figure 1. Optical flow from 50 training videos and 950 consecutive frame pairs. The 103,050 retained pixels have intensity above 0.2 at the same location in both frames and flow magnitude above 0.5 pixels/frame. Bars show fractions of retained pixels; median speed is 3.32 pixels/frame. Direction uses 18 bins of $20°$, with $0°$ right and $90°$ up.

2 Baseline ConvLSTM

The baseline has two 64-channel ConvLSTM layers and the prescribed nine-step autoregressive rollout without teacher forcing; outputs of updates 5–9 give the five forecasts.

Q2.3: Model A's reach

Pixel unshuffle changes the 64×64 image into a 16×16 grid, placing each 4×4 patch into channels. Therefore, one folded-grid position corresponds to four original pixels.

A 3×3 convolution moves information by at most one folded-grid position along each axis. Paths from the current input or the previous layer-1 hidden state traverse both ConvLSTM layers; the previous layer-2 hidden state traverses only layer 2. Thus, the longest within-step path contains two spatial convolutions.

The reach is therefore $2 \times 4 = 8$ original pixels per axis, or $8\sqrt{2} \approx 11.3$ pixels diagonally. The patch width is not added, and the 1×1 readout and shuffle operations do not increase this reach.

Table 2. Original $1\times$ forecasting errors and trainable parameters. Learned entries are mean (seed 0, seed 1); references are deterministic.

(a) First three horizons

Model	MSE_1	MSE_2	MSE_3
A	0.0170 (0.0178, 0.0163)	0.0207 (0.0212, 0.0201)	0.0230 (0.0234, 0.0226)
Blank	0.0426	0.0426	0.0426
Copy-last	0.0492	0.0604	0.0669

(b) Later horizons and mean error

Model	MSE_4	MSE_5	$E^{(5)}$
A	0.0247 (0.0250, 0.0245)	0.0265 (0.0266, 0.0263)	0.0224 (0.0228, 0.0219)
Blank	0.0425	0.0425	0.0426
Copy-last	0.0708	0.0732	0.0641

(c) Trainable parameters

Model	Parameters
A	480784 (480784, 480784)
Blank	0
Copy-last	0

Q2.4: Baseline performance

Model A outperforms Blank and Copy-last at every horizon. Its mean $E^{(5)}$ is 0.0224, compared with 0.0426 and 0.0641 respectively.

Its error increases from $\text{MSE}_1 = 0.0170$ to $\text{MSE}_5 = 0.0265$. Both seeds show this trend, consistent with accumulating autoregressive error. Blank remains approximately constant, while Copy-last becomes progressively worse.

In Figure 2, overlapping digits become blurry blobs with merging and loss of identity. However, blur is also visible without overlap, so overlap alone does not explain these failures.

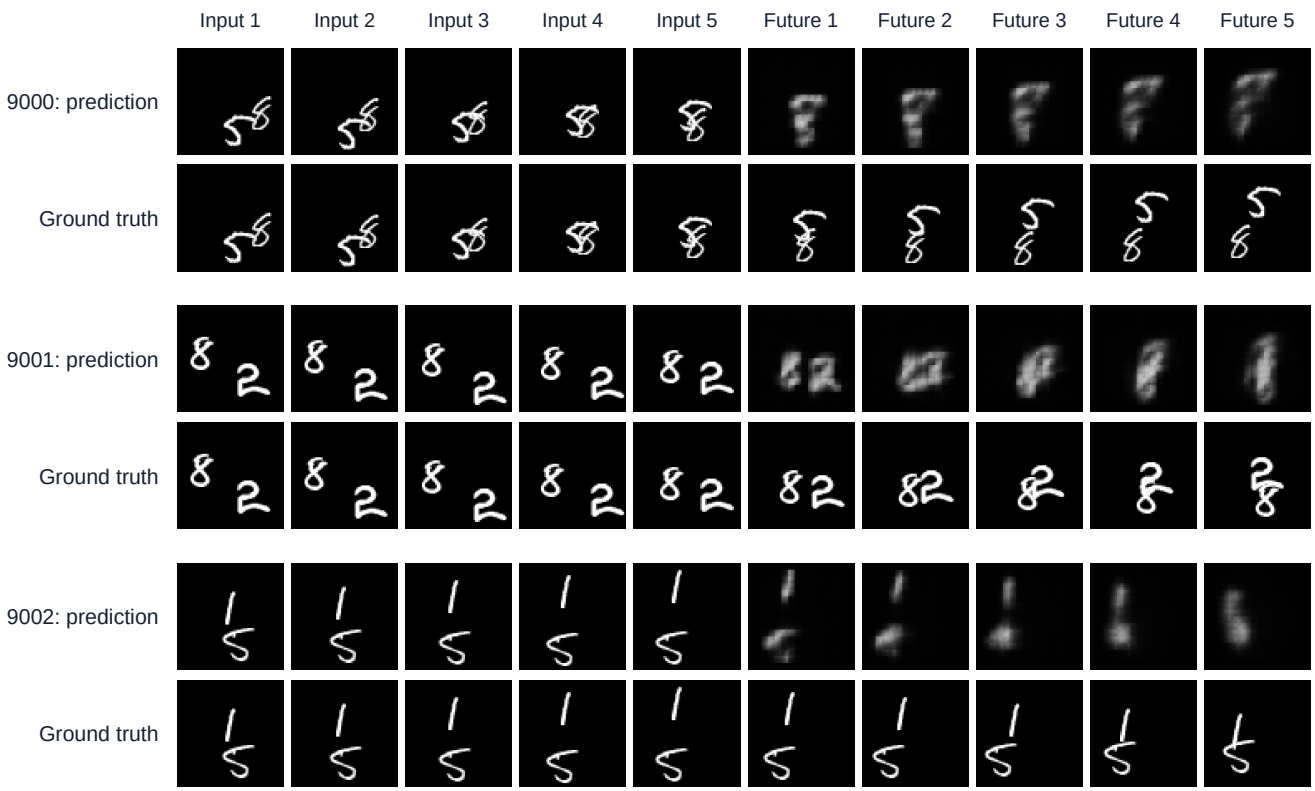


Figure 2. Model A, seed 0, final epoch 30, on original $1\times$ test videos 9000–9002. Columns 1–5 are observed frames and columns 6–10 are future frames. Each pair of rows shows predictions and ground truth. The examples include overlapping digits.

Q2.5: Predictions before testing the baseline

According to my understanding of Model A, I predict that it will fail both the mirror and rotation tests. Although Figure 1 shows broad coverage of optical flow directions, the architecture does not explicitly enforce consistency under mirroring or rotation. Thus, broad direction coverage alone may not be sufficient for consistent predictions on transformed videos.

For speed transformations, I expect noticeable degradation at 2x and the clearest breakdown at 3x. Typical training speeds are around 3-4 pixels/frame, whereas the scaled median references are approximately 6.64 and 9.95 pixels/frame. The 2x reference is within the model's eight-pixel reach along each axis, but these larger displacements are less familiar from training. At 3x, near-axis-aligned movement can exceed that reach, although some diagonal movements remain within the spatial support. Therefore, unfamiliar speeds and limited reach may both contribute to increased blur, misplaced predictions and loss of digit identity

Prediction preserved verbatim from pre-test commit `31adeaa`.

3 Testing the Baseline

Table 3. Original and transformed errors, with mirror/rotation consistency gaps at $1\times$.

(a) Forecasting errors

Model	$E^{(5)}$	E_M	E_R
A	0.0224 (0.0228, 0.0219)	0.0230 (0.0234, 0.0226)	0.0239 (0.0244, 0.0234)
Blank	0.0426	0.0426	0.0426
Copy-last	0.0641	0.0641	0.0641

(b) Consistency gaps

Model	C_M	C_R
A	0.00166 (0.00165, 0.00168)	0.00286 (0.00283, 0.00289)
Blank	0.00×10^{0}	0.00×10^{0}
Copy-last	0.00×10^{0}	0.00×10^{0}

Q2.6: Consistency gap and error

Reading and evaluating on the basis of consistency gap alone is problematic because transformation equivariance does not guarantee accurate forecasts. Blank and Copy-last both have $C_T = 0$, yet their transformed errors are 0.0426 and 0.0641, respectively. Thus, E_T needs to be reported alongside C_T to check whether the forecasts also match the ground truth.

Q2.7: Video 9925 observations

In Figure 3(d), video 9925 has a rotation consistency gap of approximately 0.00749. In row 3, where the model receives the rotated inputs, the predicted digits stretch vertically towards each other and partly merge into a blur at later horizons. In row 4, where rotation is applied to the original forecasts, the digits remain separate, although their shapes are still blurry.

Thus, rotation changes the model's prediction of the digits' relative motion and separation. Row 4 preserves separation better, but this does not establish that its motion is correct.

Q2.8: Transformation comparison

Table 3 indicates that rotation is harder for model A: $C_R = 0.00286$, compared with $C_M = 0.00166$, while $E_R = 0.0239$, compared with $E_M = 0.0230$. Both seeds show this trend.

From Q2.1, motion directions are broadly covered but not equally common. From Q2.2, both transformations preserve speed while changing motion direction and digit appearance. Unfamiliar rotated appearances are therefore a plausible contributor, although direction bias may also contribute. These results alone cannot isolate the two effects.

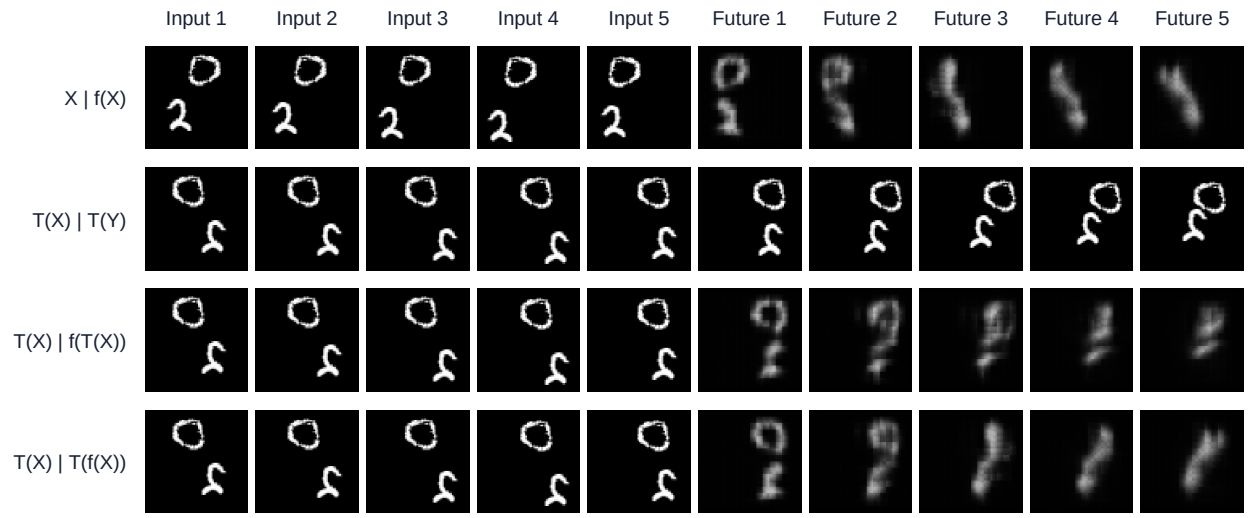


Figure 3(a). Model A, seed 0, $1\times$, video 9257, transformation M. $C_M = 0.00456$, $E_M = 0.0311$. The two largest per-video gaps were selected separately for each transformation.
GIF: outputs/gifs/figure3a_M_video9257.gif.

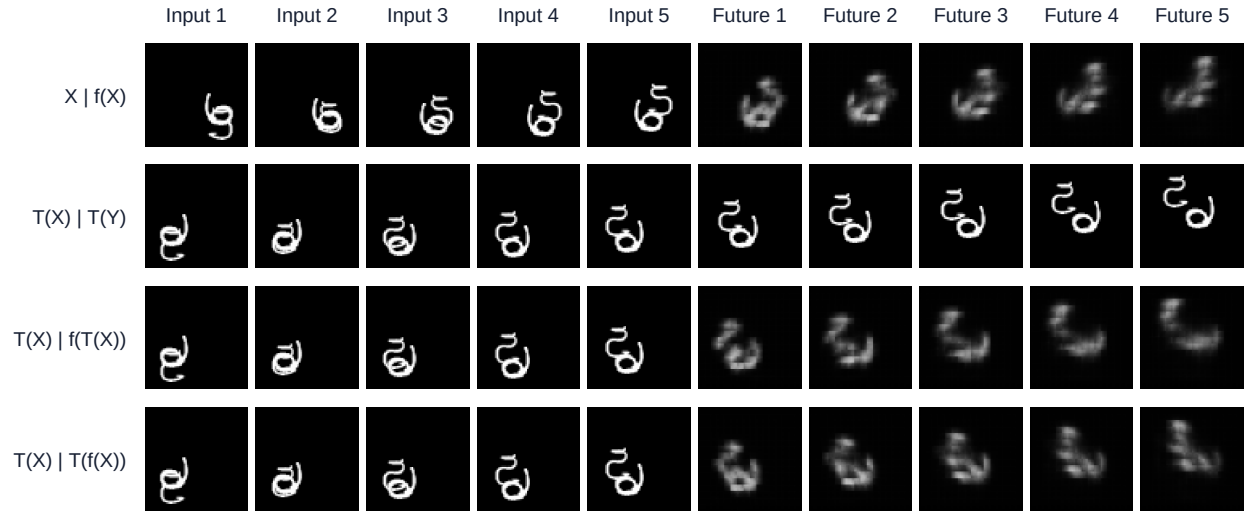


Figure 3(b). Model A, seed 0, $1\times$, video 9348, transformation M. $C_M = 0.00436$, $E_M = 0.0237$. The two largest per-video gaps were selected separately for each transformation.
GIF: outputs/gifs/figure3b_M_video9348.gif.

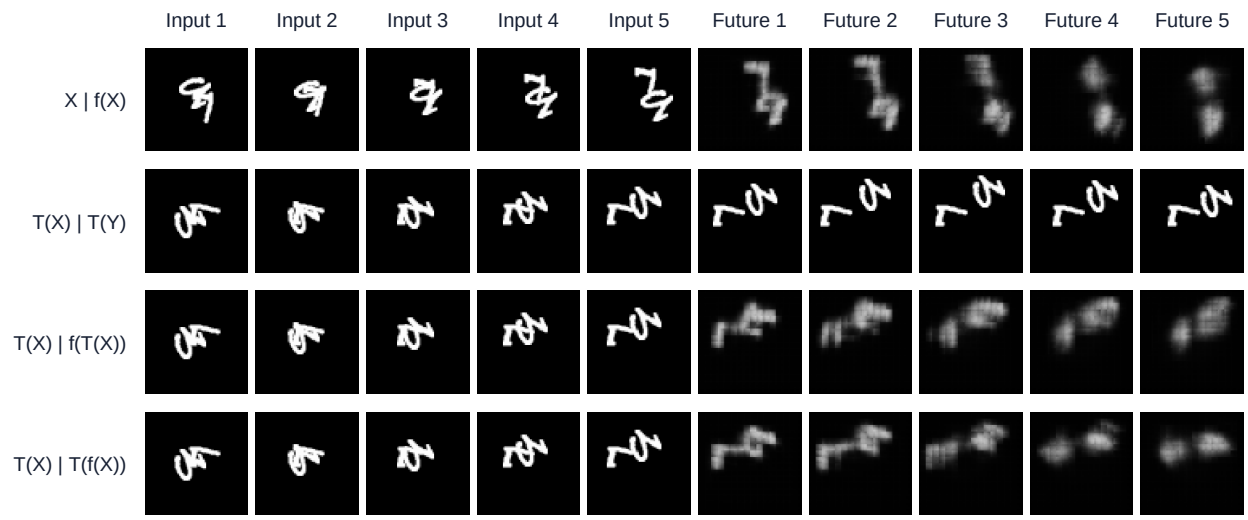


Figure 3(c). Model A, seed 0, $1\times$, video 9497, transformation R. $C_R = 0.00792$, $E_R = 0.0331$. The two largest per-video gaps were selected separately for each transformation.
GIF: outputs/gifs/figure3c_R_video9497.gif.

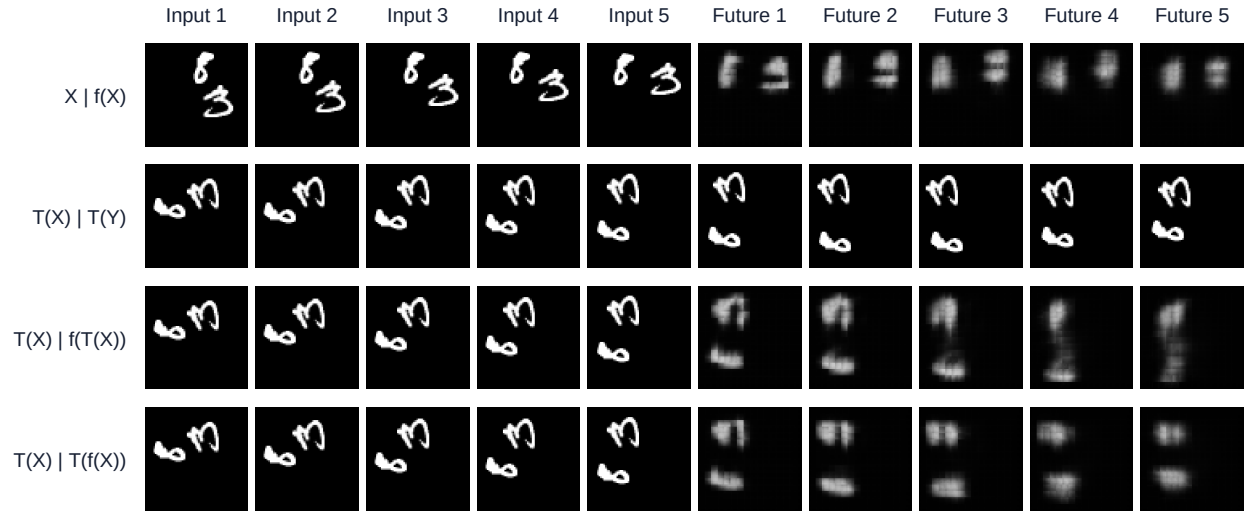


Figure 3(d). Model A, seed 0, $1\times$, video 9925, transformation R. $C_R = 0.00749$, $E_R = 0.0364$. The two largest per-video gaps were selected separately for each transformation.
GIF: outputs/gifs/figure3d_R_video9925.gif.

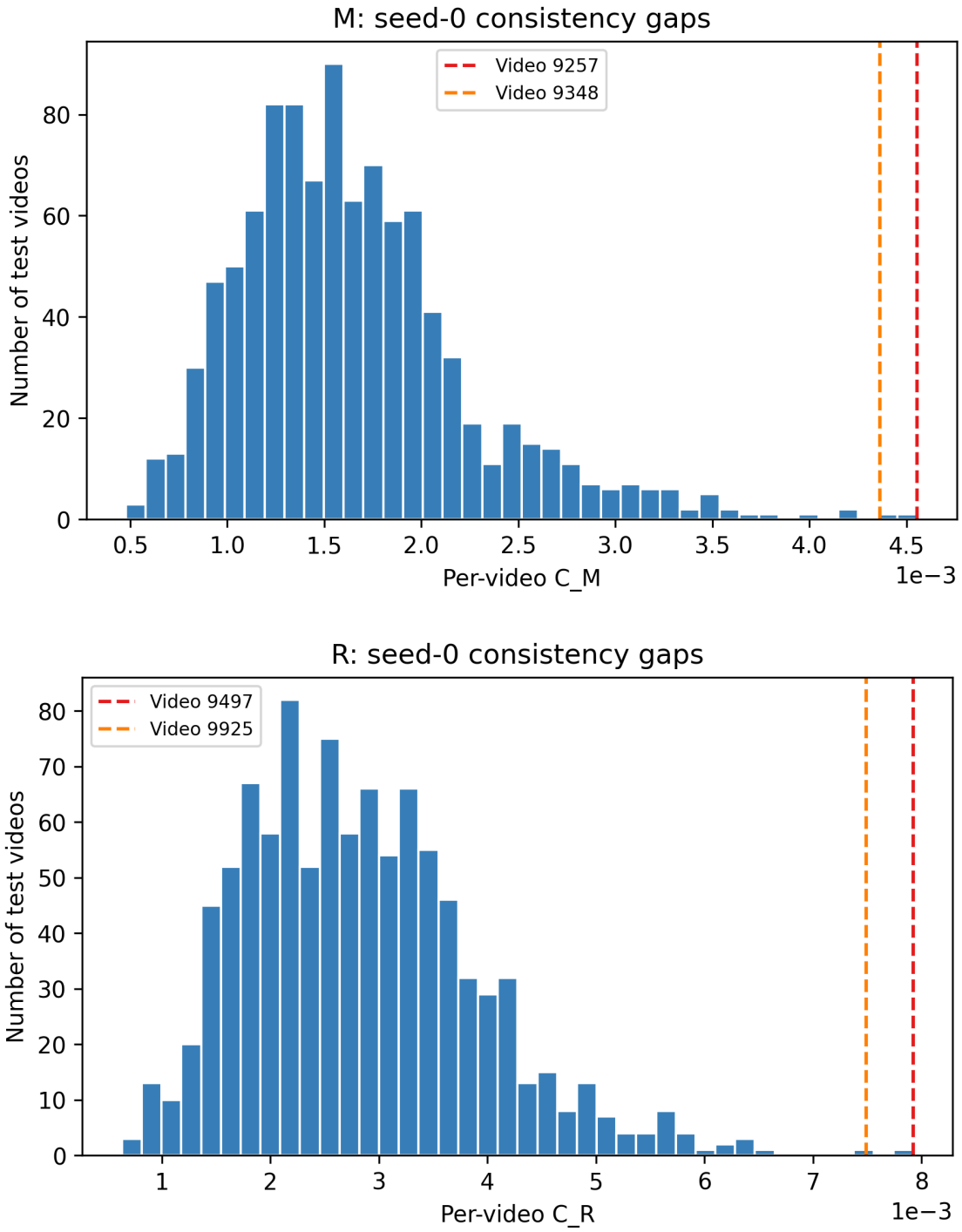


Figure 4. Per-video mirror and rotation consistency gaps for seed-0 model A on all 1,000 original $1\times$ test windows. Lines mark the Figure 3 cases: 9257 and 9348 for M, and 9497 and 9925 for R.

Faster motion

Table 4. Baseline speed tests. Cross-speed comparisons use exactly two forecast horizons.

(a) Two-horizon error

Model	$E^{(2)}, 1\times$	$E^{(2)}, 2\times$	$E^{(2)}, 3\times$
A	0.0189 (0.0195, 0.0182)	0.0443 (0.0436, 0.0451)	0.0503 (0.0499, 0.0506)
Copy-last	0.0548	0.0661	0.0703

(b) Five-horizon error where targets exist

Model	$E^{(5)}, 1\times$	$E^{(5)}, 2\times$
A	0.0224 (0.0228, 0.0219)	0.0482 (0.0475, 0.0489)
Copy-last	0.0641	0.0719

Q2.9: Faster motion and architectural reach

Model A shows substantial degradation at $2\times$ and becomes worse at $3\times$: $E^{(2)}$ increases from 0.0189 to 0.0443 and 0.0503, respectively. Figure 5 already shows blur at $1\times$; at faster speeds, digit shapes deteriorate further. At $3\times$, the predicted 4 changes little between forecasts while its ground-truth position continues changing. This suggests underestimated motion, while the blur suggests uncertainty about shape and position.

The degradation at $2\times$ occurs despite its median speed reference of 6.64 pixels/frame being below the estimated eight-pixel reach per axis. Thus, reach is an upper bound on information movement, not a guarantee of successful forecasting. The estimate does not capture unfamiliar training speeds, and a median does not bound every displacement.

Q2.10: Separating unfamiliar motion from limited reach

These results alone cannot distinguish the two explanations. We could train the same architecture on mixed $1\times$, $2\times$ and $3\times$ windows, then compare it with a larger-reach model trained on the same mixture. Training budgets and evaluation should remain fixed, with comparable parameter counts.

Improvement from speed augmentation would show that unfamiliar motion contributes. Further improvement from the larger-reach model would support architectural reach also being a limitation.

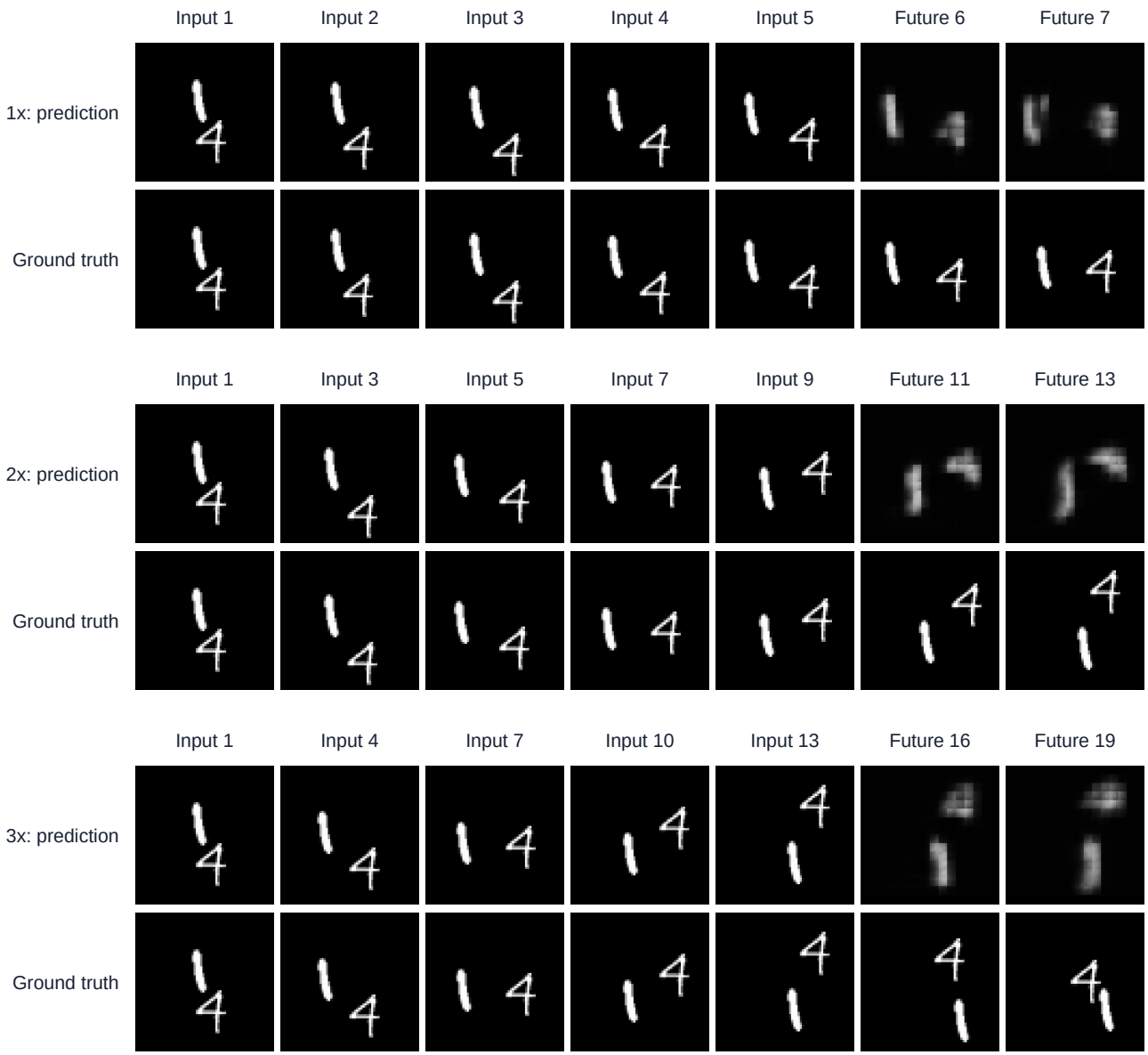


Figure 5. Model A, seed 0, test video 9021. Each speed shows five inputs and the first two predicted/true future frames. Header numbers refer to frames in the original video. The digits do not overlap in any displayed true frame.

4 What Can Data Fix?

Q2.11: Predictions before training B–D

I expect model B to significantly resolve transformation equivariance and reduce the transformation consistency gaps. I expect model C to improve the variable speed forecasts since the same architecture is exposed uniformly to varying motions. Model D's larger reach due to dilation may improve the faster-speed forecasts further.

Prediction preserved verbatim from pre-training commit `2e5b8f5`.

Q2.12: Model D and its reach

Model D keeps the baseline's gate equations and nine-step rollout, but changes the second layer to a 3×3 convolution with dilation 2 and padding 2. Its sampled offsets are $-2, 0, 2$ rather than $-1, 0, 1$, while the folded grid remains 16×16.

Paths from the current input or previous layer-1 hidden state traverse both convolutions, giving $(1 + 2) \times 4 = 12$ pixels per axis. The previous layer-2 hidden state traverses only its dilation-2 convolution, giving 8 pixels. The longest reach is therefore 12 pixels per axis, or $12\sqrt{2} \approx 17.0$ pixels diagonally, strictly larger than A's reach.

Dilation adds no weights. D therefore retains 480,784 parameters, below the 721,176 limit.

B retains A's architecture and $1\times$ training, adding uniform square-symmetry augmentation. C retains A's architecture and samples speeds uniformly from $\{1, 2, 3\}$; D uses the same speed protocol with the larger reach described above. C and D sample starts 1–14, 1–8 and 1–2 at the three speeds respectively.

Table 5. Models A–D: errors, consistency, parameter count and measured cost. Each cell gives the mean above (seed 0, seed 1).

(a) Original/transformed errors and consistency gaps

Model	$E^{(5)}, 1\times$	E_M	E_R	C_M	C_R
A	0.0224 (0.0228, 0.0219)	0.0230 (0.0234, 0.0226)	0.0239 (0.0244, 0.0234)	0.00166 (0.00165, 0.00168)	0.00286 (0.00283, 0.00289)
B	0.0227 (0.0236, 0.0217)	0.0226 (0.0235, 0.0218)	0.0227 (0.0236, 0.0218)	0.00121 (0.00118, 0.00124)	0.00133 (0.00125, 0.00142)
C	0.0270 (0.0275, 0.0264)	0.0272 (0.0277, 0.0267)	0.0280 (0.0285, 0.0275)	8.63×10^{-4} (8.74×10^{-4}, 8.52×10^{-4})	0.00183 (0.00183, 0.00183)
D	0.0272 (0.0271, 0.0274)	0.0274 (0.0273, 0.0275)	0.0284 (0.0283, 0.0284)	9.18×10^{-4} (0.00109, 7.45×10^{-4})	0.00211 (0.00232, 0.00191)

(b) Two-horizon speed errors, parameters and cost

Model	$E^{(2)}, 1\times$	$E^{(2)}, 2\times$	$E^{(2)}, 3\times$	Params	Train s/epoch	Inference s/1,000
A	0.0189 (0.0195, 0.0182)	0.0443 (0.0436, 0.0451)	0.0503 (0.0499, 0.0506)	480784	6.10 (6.11, 6.09)	0.195 (0.187, 0.202)
B	0.0190 (0.0199, 0.0182)	0.0443 (0.0435, 0.0452)	0.0509 (0.0505, 0.0513)	480784	6.54 (6.56, 6.52)	0.188 (0.190, 0.187)
C	0.0231 (0.0238, 0.0225)	0.0291 (0.0292, 0.0290)	0.0333 (0.0335, 0.0332)	480784	6.21 (6.20, 6.22)	0.191 (0.191, 0.191)
D	0.0245 (0.0243, 0.0247)	0.0288 (0.0286, 0.0290)	0.0325 (0.0324, 0.0325)	480784	6.21 (6.21, 6.21)	0.190 (0.192, 0.188)

Q2.13: Model failures

Failures under mirroring and rotation appear partly appearance related: compared with A, B's consistency gaps decrease (C_M from 0.00166 to 0.00121 and C_R from 0.00286 to 0.00133). The transformed errors also decrease: E_M from 0.0230 to 0.0226 and E_R from 0.0239 to 0.0227. However, augmentation also changes direction coverage, so this comparison cannot isolate appearance completely.

The unfamiliar motion failures are the faster speed failures. With the same architecture, C reduces $E^{(2)}$ from 0.0443 to 0.0291 at $2\times$ and from 0.0503 to 0.0333 at $3\times$. Thus, exposure to faster motion substantially improves forecasting.

There is weaker evidence for architectural reach being the main limitation here. D further reduces the $3\times$ error to 0.0325, but this is a modest improvement over C, rather than C failing and D fully resolving the failure. More training data will carry more distributional information thereby improving the overall learned weights but cannot change the reach which is primarily dependent on the spatial connections (i.e. architecture).

Q2.14: Accuracy on original clips

These changes do cost some accuracy on the original $1\times$ clips. B increases mean $E^{(5)}$ slightly, from A's 0.0224 to 0.0227, although the change is mixed across seeds. C and D increase it to 0.0270 and 0.0272, respectively.

For handling varying speeds, this trade-off seems worthwhile because the faster-speed errors decrease substantially. However, A remains preferable if performance on original $1\times$ clips is the only objective.

5 The Final Model

The final model is symmetrised D, the prescribed default. D gives the lowest mean $3\times$ error in Table 5. No optional D+B run or additional training is used. All eight transformed branches run the complete forecaster before inverse transformation and averaging; they share one set of weights. The group is $G = \{R^a, MR^a : a = 0, 1, 2, 3\}$; MR^a rotates first and then mirrors.

$$\bar{f}(X) = \frac{1}{8} \sum_{g \in G} g^{-1} f(gX).$$

For $h \in G$, set $q = gh$. Right multiplication permutes the group and $g^{-1} = hq^{-1}$, so

$$\bar{f}(hX) = \frac{1}{8} \sum_{q \in G} hq^{-1} f(qX) = h\bar{f}(X).$$

A mirror/rotation is a linear pixel permutation, so it commutes with averaging. Thus $C_M = C_R = 0$ in exact arithmetic for any base forecaster. The measured floating-point gaps are retained in Table 6.

Table 6. Symmetrised B and D; symmetrised D is the final model. Gaps are measured floating-point values. Training time belongs to the underlying model. Each cell gives the mean above (seed 0, seed 1).

(a) Original/transformed errors and consistency gaps

Model	$E^{(5)}$, $1\times$	E_M	E_R	C_M	C_R
Sym B	0.0221 (0.0231, 0.0212)	0.0221 (0.0231, 0.0212)	0.0221 (0.0231, 0.0212)	4.29×10^{-17} (3.93×10^{-17}, 4.64×10^{-17})	4.40×10^{-17} (4.04×10^{-17}, 4.76×10^{-17})
Sym D (final)	0.0272 (0.0270, 0.0273)	0.0272 (0.0270, 0.0273)	0.0272 (0.0270, 0.0273)	3.34×10^{-17} (3.65×10^{-17}, 3.04×10^{-17})	3.44×10^{-17} (3.76×10^{-17}, 3.12×10^{-17})

(b) Two-horizon speed errors, parameters and cost

Model	$E^{(2)}$, $1\times$	$E^{(2)}$, $2\times$	$E^{(2)}$, $3\times$	Params	Train s/epoch	Inference s/1,000
Sym B	0.0186 (0.0195, 0.0178)	0.0435 (0.0428, 0.0442)	0.0498 (0.0496, 0.0500)	480784	6.54 (6.56, 6.52)	1.49 (1.48, 1.50)
Sym D (final)	0.0245 (0.0243, 0.0247)	0.0288 (0.0286, 0.0290)	0.0324 (0.0323, 0.0325)	480784	6.21 (6.21, 6.21)	1.49 (1.49, 1.48)

Q2.15: Learned and built-in symmetry

Model B learns approximate consistency through augmentation, but its gaps remain nonzero: $C_M = 0.00121$ and $C_R = 0.00133$. Built-in symmetry is exact mathematically because transforming the input only reorders the eight aligned forecasts. The measured gaps are around 10^{-17}, reflecting floating-point rounding.

Original and transformed errors improved slightly after wrapping. B's mean original $E^{(5)}$ decreased from 0.0227 to 0.0221. D also improved slightly in full precision, although both values round to 0.0272. Inference time increased from approximately 0.19 to 1.49 seconds per 1,000 videos, roughly eight times, with no additional training.

Averaging can hurt accuracy when branches disagree about digit positions and smear an otherwise good forecast. Here, the mean errors decreased despite the visible blur.

Q2.16: Returning to video 9925

In Figure 6(a4), rows 3 and 4 now agree, with $C_R \approx 5.79\times10^{-17}$. However, the predictions are still blurry and indistinguishable blobs, whereas the true digits remain separate. Thus, the final model does not fully get this video right.

Symmetrisation reduces D's error on this clip from 0.0430 to 0.0388, but this remains worse than A's 0.0364 and B's 0.0316. The wrapper fixed the consistency failure without fixing digit identity and tracking. B performs best on this example; the faster-motion improvements from C and D do not produce a better forecast of this particular $1\times$ clip.

Q2.17: Final verdict

The final model has learned some useful motion patterns, but not a reliable general motion rule. Its $E^{(2)}$ at $3\times$ decreases from model A's 0.0503 to 0.0324, at a cost to original $1\times$ accuracy. However, Figure 6 still shows blurry predictions and loss of digit identity. Exact mirror/rotation consistency is guaranteed by the wrapper and therefore does not establish learned motion. Stronger evidence would be sharp forecasts that maintain digit identity and correct trajectories through overlaps and bounces under unseen conditions. Generalisation to speeds beyond the trained $1\times$-$3\times$ range remains uncertain.

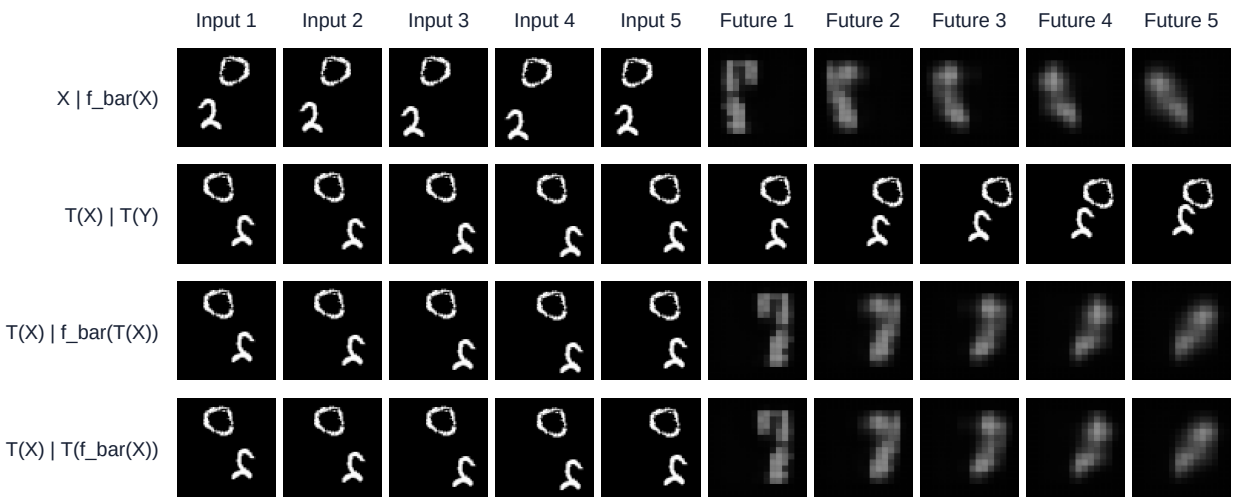


Figure 6(a1). Final model: symmetrised D, seed 0, $1\times$, video 9257, transformation M. $C_M = 4.90\times10^{-17}$, $E_M = 0.0331$. The video is the same baseline Figure 3 case; rows 3 and 4 agree up to floating-point error.
GIF: outputs/gifs/figure6a1_M_video9257.gif.



Figure 6(a2). Final model: symmetrised D, seed 0, $1\times$, video 9348, transformation M. $C_M = 3.84\times10^{-17}$, $E_M = 0.0269$. The video is the same baseline Figure 3 case; rows 3 and 4 agree up to floating-point error.
GIF: outputs/gifs/figure6a2_M_video9348.gif.

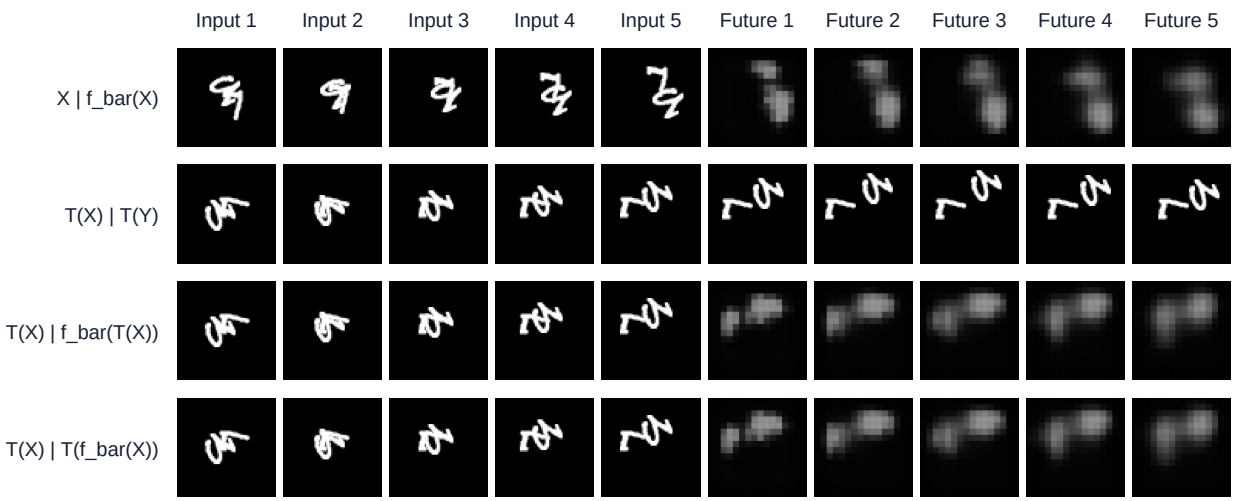


Figure 6(a3). Final model: symmetrised D, seed 0, $1\times$, video 9497, transformation R. C_R = 5.52×10^{-17}, $E_R = 0.0338$. The video is the same baseline Figure 3 case; rows 3 and 4 agree up to floating-point error.
GIF: outputs/gifs/figure6a3_R_video9497.gif.

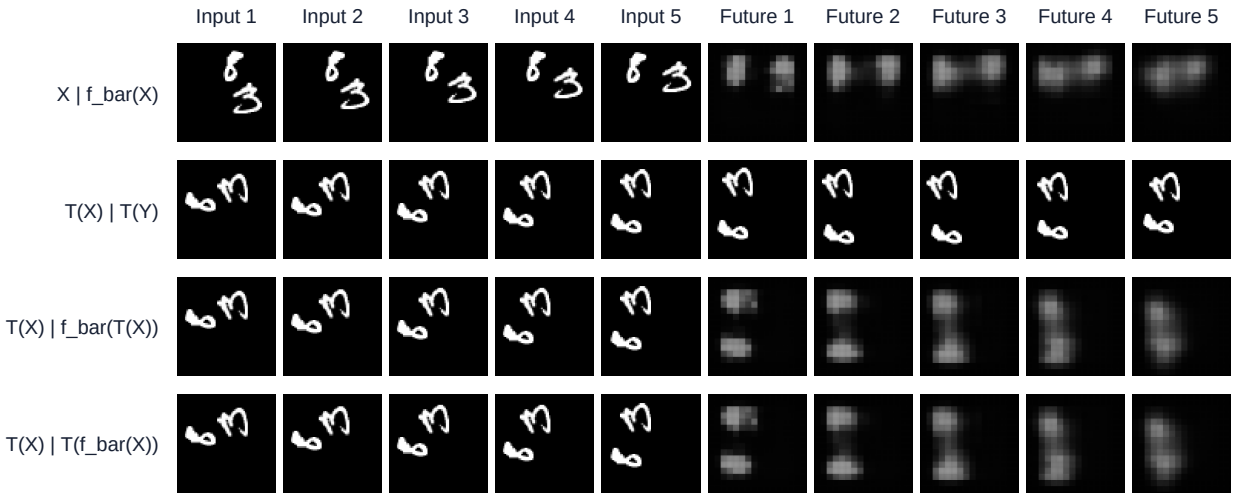


Figure 6(a4). Final model: symmetrised D, seed 0, $1\times$, video 9925, transformation R. C_R = 5.79×10^{-17}, $E_R = 0.0388$. The video is the same baseline Figure 3 case; rows 3 and 4 agree up to floating-point error.
GIF: outputs/gifs/figure6a4_R_video9925.gif.

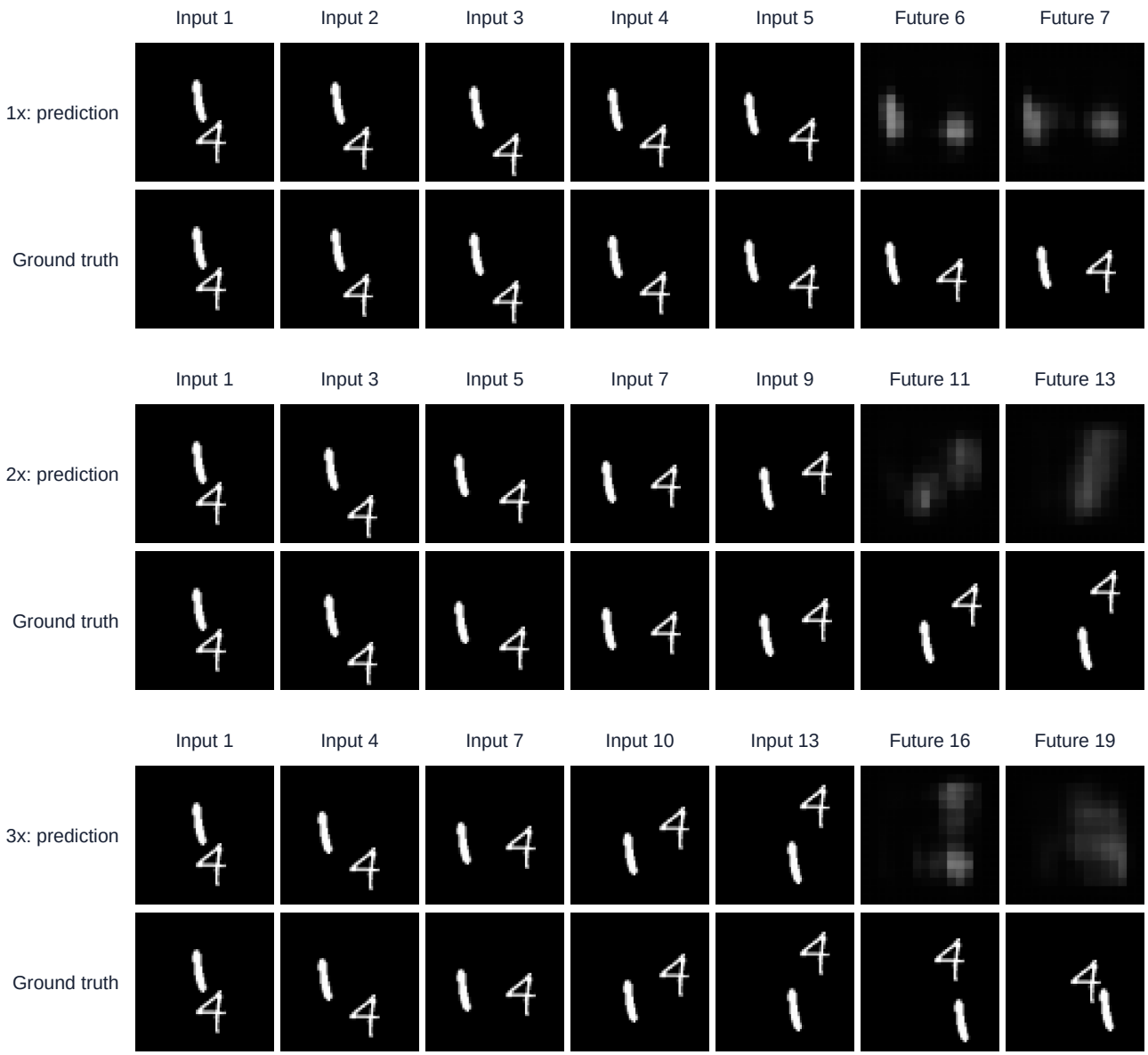
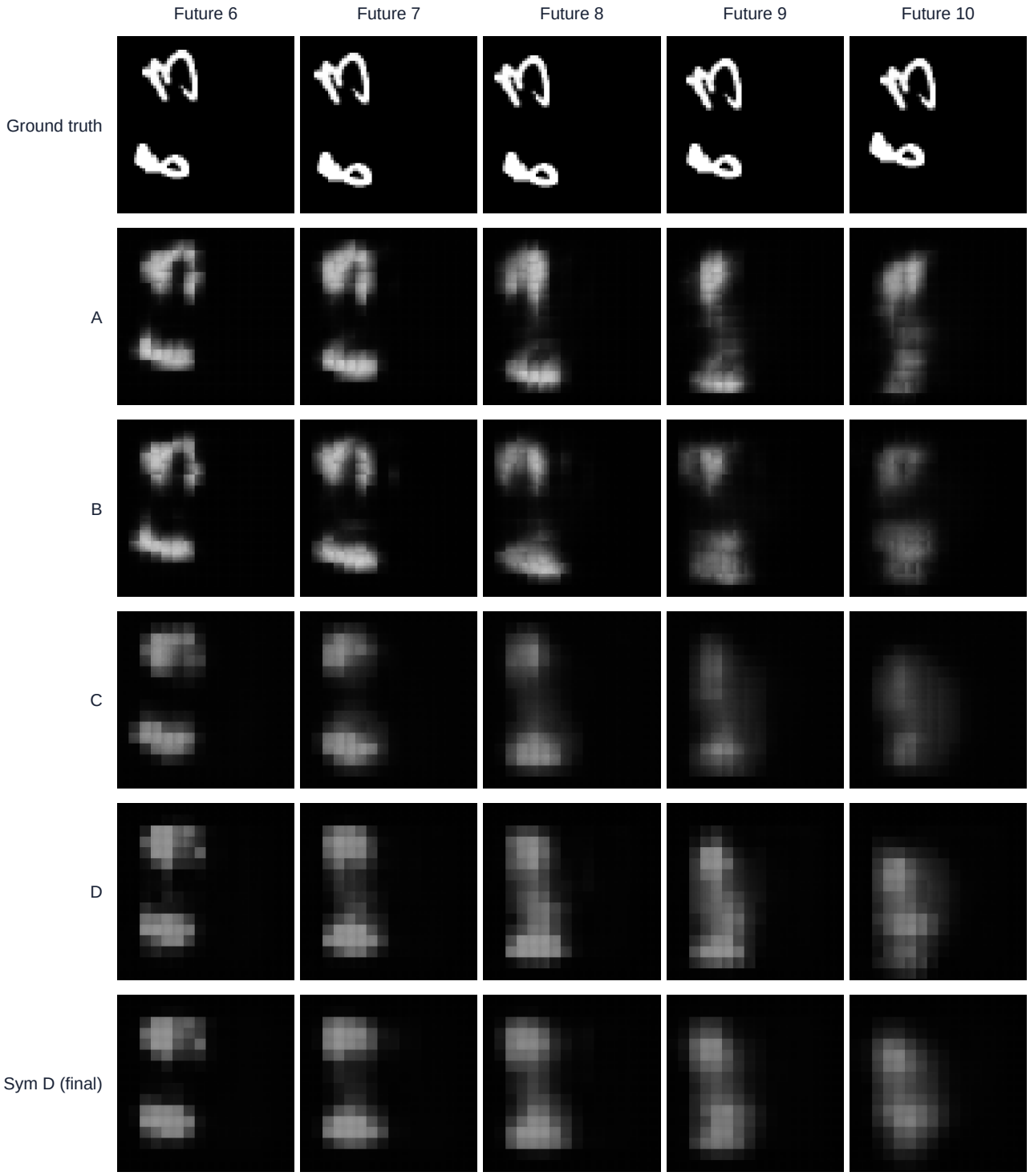


Figure 6(b). Final model: symmetrised D, seed 0, original test video 9021. This repeats Figure 5 on the same video, using the same five inputs and first two future frames at each speed. Header numbers identify original-video frames.

Supplementary comparison for Q2.16



Video 9925 diagnostic. Rotated video 9925, seed 0, future frames 6–10. The ground truth and A/B/C/D/final forecasts show that enforcing consistency does not restore sharp digit identity or guarantee correct motion.

Generative AI Usage Declaration

All of the work in this report has been designed, attempted, reviewed, and finalized by me—the author. The author does acknowledge the use of coding agents like Codex to generate and refine code for Q2; as well as for compiling and formatting the diagrams and noted observations and answers from the Jupyter notebook into a LaTeX file.